

INTELLIGENT JOB PORTAL

A Java-Based Intelligent Job Matching System Using Data Structures and Algorithms

Domain: Data Structures & Algorithms

Platform: Core Java (Swing UI, File Handling)

Team: Abhay , Pranav(2520030353,2520030477)

Guide: V.Sireesha

Department: Computer Science & Engineering

Academic Year: 2025 – 2026

Basic idea:



Problem Statement

- Traditional job portals list thousands of jobs — candidates must search manually.
- Only a small percentage of listings are actually relevant to a given profile.
- Plain keyword search often returns irrelevant or loosely related results.
- Skills can be written in different forms (e.g. “JavaScript” vs “Java Script”).
- Manually comparing candidate skills with job requirements takes a lot of time.
- Ranking jobs by suitability is difficult without an efficient method.

Did you know?

A single job portal can host over 50,000 active listings, but a candidate may qualify for only a few dozen of them.

Proposed idea: Our Intelligent Job Portal uses Data Structures and Algorithms to search, compare, match and rank jobs according to a candidate's profile.

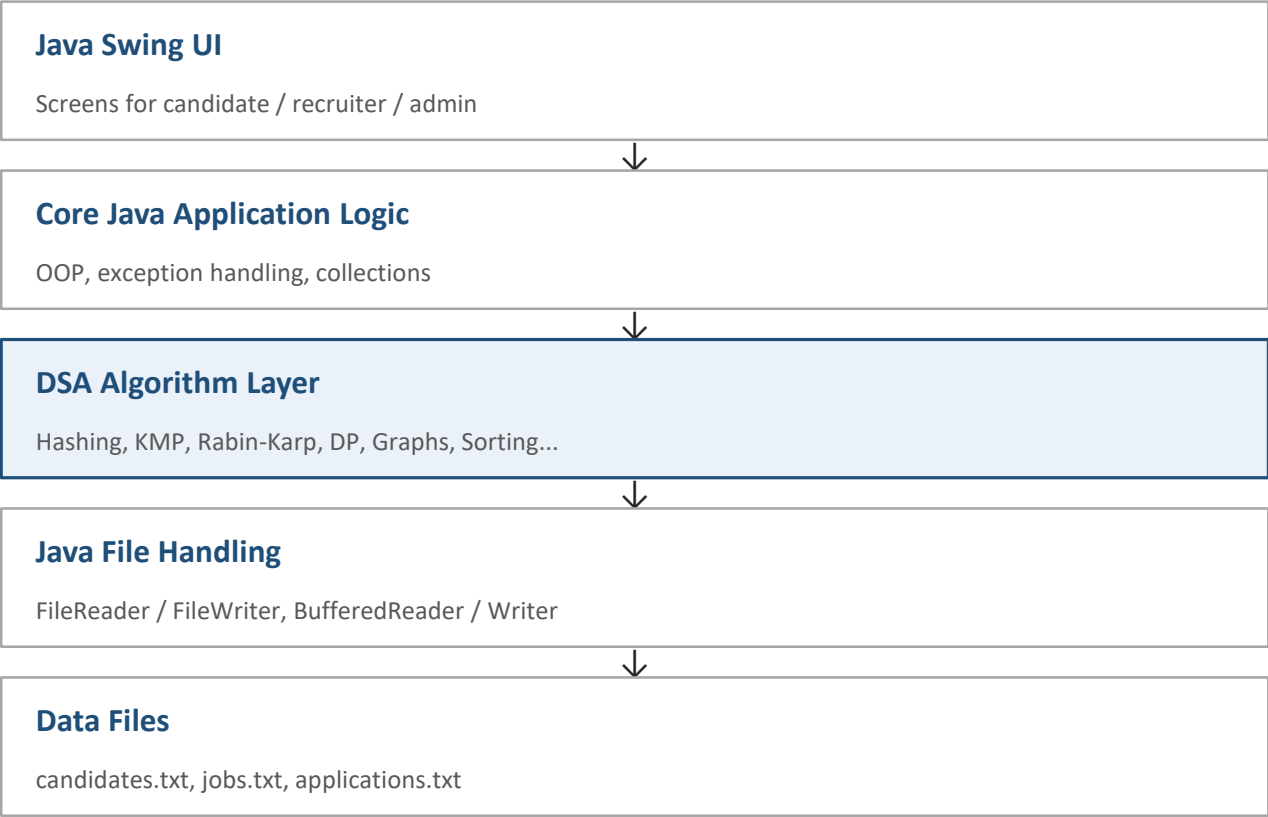
Project Objectives

1. Develop a job portal application using Core Java.
2. Use Java File Handling as the backend for persistent data storage.
3. Allow candidates to create profiles with skills, qualifications and experience.
4. Allow recruiters / admins to create and manage job listings.
5. Apply string algorithms to search and compare skills.
6. Use DSA techniques to calculate job relevance and ranking.
7. Demonstrate practical use of concepts from the DSA syllabus.

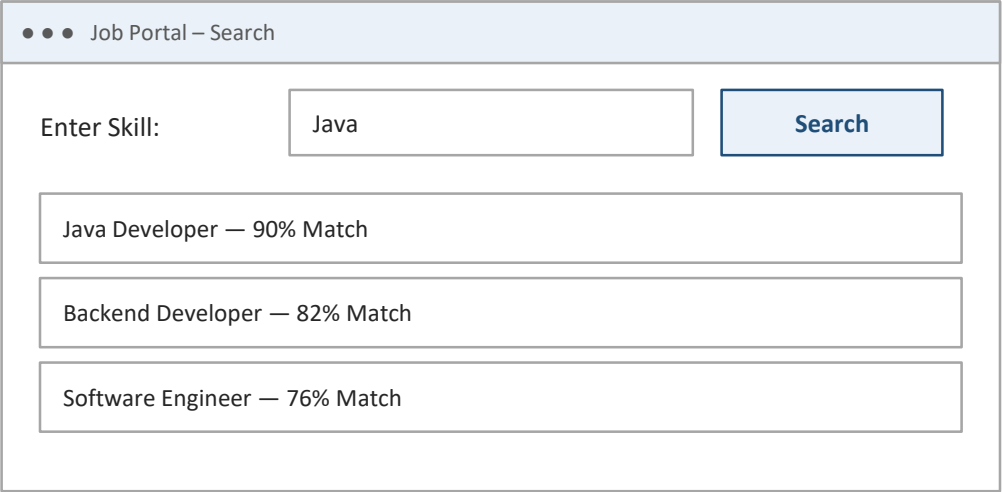
How it fits together:



System Components & Technology



Note: No MySQL or external database is used — all data is stored and read using plain Java file handling (.txt / .csv).



(illustrative Swing UI mockup)

Technologies used

- Java Swing – desktop user interface
- Core Java – OOP, collections, exceptions
- DSA Layer – hashing, string search, graphs, sorting
- Java File Handling – .txt based storage (no database)

Data Structures Used

Data Structure	Where It Is Used	Complexity
ArrayList	Store candidate and job records	$O(1)$ access
HashMap	Fast skill / profile lookup by key	$O(1)$ average
HashSet	Store unique skills (avoid duplicates)	$O(1)$ average
Priority Queue	Maintain top job recommendations	$O(\log n)$ insert
Graph	Candidate – Skill – Job relationships	$O(V + E)$ traversal
String / Char Array	Process job descriptions and skill text	$O(n)$ scan

Complexity in short:

HashMap lookup $\rightarrow O(1)$ average Priority Queue insertion $\rightarrow O(\log n)$ Graph traversal $\rightarrow O(V + E)$

Intelligent Skill & Keyword Searching

KMP – search one skill inside a job description

Search skill: **“Java”**

Job text: “Looking for Java developer with DSA and SQL knowledge.”

KMP scans the text once and avoids re-checking characters already matched — faster than a plain nested-loop search.

Aho-Corasick – search multiple skills at once

Java

Python

SQL

DSA

Spring

One pass through the job description checks for all required skills together, instead of searching each skill separately.

Rabin-Karp – hashing-based alternative

Compares hash values of the skill and text windows instead of characters directly — useful when searching the same job text for many different skills.

Edit Distance

Handles slightly different spellings of the same skill.

“Java Script”

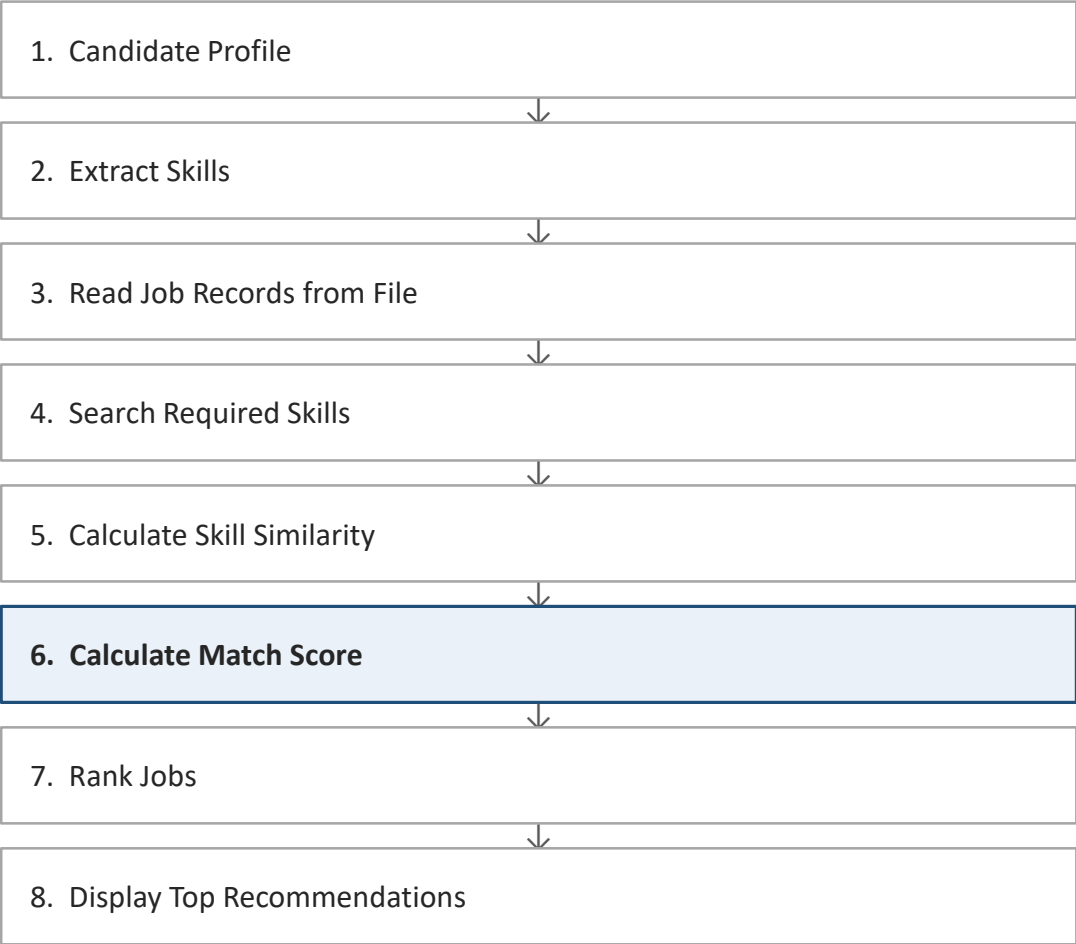
↕ 1 edit

“JavaScript”

If the edit distance is small, the two skills are treated as the same skill while matching.

Where used: skill normalisation before the matching step, so minor spelling differences don't reduce the match score.

Intelligent Job Matching Algorithm



Worked Example

Candidate has: **Java, SQL, Python, DSA**

Job A requires: **Java, SQL, DSA, Spring**

Matching skills: **Java + SQL + DSA = 3 / 4**

Match Score = 3 / 4 = 75%

Job A	75% Match
Job B	60% Match
Job C	45% Match

Highest-ranked jobs are displayed first.

Dynamic Programming & Graph Application

Part 1 – Dynamic Programming (Edit Distance)

- Used to check how closely a candidate skill matches a required skill.
- DP avoids solving the same character-comparison subproblems again.
- A small edit-distance value means the two skills are treated as a match.

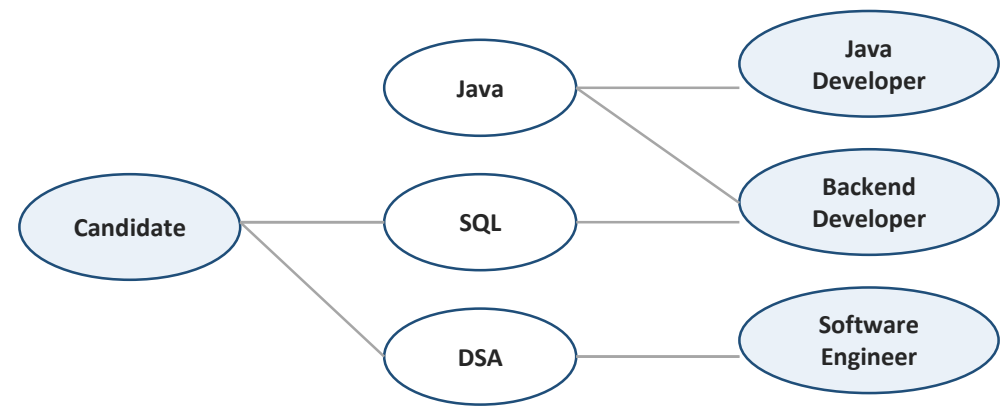
Comparing: “JavaScript” vs “Javascript”

	J	a	v	a	S
J	0	1	2	3	4
a	1	0	1	2	3

(sample DP table – minimum edits required)

Part 2 – Graph (Candidate – Skill – Job)

- BFS / DFS explores jobs reachable from a candidate’s skill set.
- Graph form makes it easy to see which jobs share common skills.



Skills act as connecting nodes between one candidate and several jobs.

Job Ranking, File Handling & Complexity

Job Ranking (Priority Queue / Sorting)

1	Java Developer	90%
2	Backend Developer	82%
3	Software Engineer	76%
4	Python Developer	68%

File Handling Backend

candidates.txt	jobs.txt	applications.txt	skills.txt
----------------	----------	------------------	------------

- FileReader / BufferedReader – read records
- FileWriter / BufferedWriter – write records
- Simple file update on each new application

Complexity Analysis

Operation	Complexity
Hashing (lookup)	$O(1)$ average
KMP search	$O(n + m)$
Sorting	$O(n \log n)$
BFS / DFS	$O(V + E)$
Priority Queue insert	$O(\log n)$

Complexity analysis helps evaluate how efficiently the job portal can search and rank jobs as data grows.

Conclusion & Future Scope

Conclusion

The Intelligent Job Portal shows how DSA concepts apply to a real recruitment problem. The system combines Core Java, File Handling, Data Structures and Algorithms to provide:

- Efficient searching
- Skill matching
- Job ranking
- Relevant recommendations
- Persistent data management

Future Scope

- Resume parsing
- Natural Language Processing
- Machine-learning-based recommendations
- Skill-gap analysis
- Personalized career suggestions
- Advanced candidate-job matching
- Online deployment
- Mobile application

“From searching jobs to intelligently matching opportunities — DSA makes the process smarter and more efficient.”